SGLang wins the GPT-OSS-120B inference framework showdown

SGLang v0.5.9 is the only framework that satisfies all four requirements — 128K context, working prefix caching, reliable Harmony-format tool calling, and LangChain compatibility — without patches or workarounds. vLLM's Chat Completions API remains broken for GPT-OSS tool calling (issue #22578, closed NOT_PLANNED), GitHub which disqualifies it for LangGraph agent workloads. llama.cpp is viable but carries an unresolved conflict between sliding-window prefix caching and memory constraints. The critical caveat across all frameworks: **128K context on a single A100-80GB is theoretically possible but practically untested** — every real deployment caps single-GPU context at ~32K, with 128K requiring TP=2 across two A100s.

GPT-OSS-120B (OpenAI, August 2025) LM Studio is a **117B-parameter MoE model** OpenAI +4 with 128 experts, Semaphore top-4 routing, Unsloth AI vLLM 5.1B active parameters per token, OpenAI Hugging Face and MXFP4-quantized weights Laozhang +2 totaling ~63GB. X Its hybrid attention architecture NVIDIA — 18 full-attention layers alternating with 18 sliding-window layers (window=128) Medium +2 — produces an unusually small KV cache of only **~4.5GB at 128K context** thanks to GQA (8 KV heads, head_dim=64). vLLM This architecture creates framework-specific challenges: the sliding-window metadata triggers ring-buffer mode in llama.cpp, and the hybrid attention pattern caused a 0% cache-hit bug in vLLM. GitHub

The comparison matrix tells a clear story

Requirement	vLLM v0.15.1	SGLang v0.5.9	llama.cpp ~b8099
128K context, single A100-80GB	⚠️ Fits in theory (~70-75GB), shown only at TP=8 in docs	⚠️ Same memory math; TP≥2 recommended	⚠️ Needs `--swa-full` + KV quantization
Working prefix caching	⚠️ SHA256 default ✅, but hybrid attention bug #32802 unfixed	✅ RadixAttention — no hash collisions by design	❌ `--swa-full` required, active bugs, heavy memory cost
Harmony tool calling via Chat Completions	❌ Broken (#22578 stale). Only `/v1/responses` works	✅ `--tool-call-parser gpt-oss` produces standard `tool_calls`	⚠️ Built-in template works, but GBNF grammar conflicts with tool calling
LangChain ChatOpenAI drop-in	❌ ChatOpenAI requires Chat Completions, which is broken	✅ Standard `/v1/chat/completions` with proper `tool_calls` objects	✅ llama-server exposes compatible API
Production maturity	✅ Mature, widely deployed	✅ 400K+ GPUs, PyTorch ecosystem GitHub Sglang	✅ 92K+ stars, rolling releases
Overall verdict	**Disqualified** — tool calling broken	**Recommended**	**Viable fallback** with trade-offs

vLLM's two unfixable bugs make it a non-starter

vLLM v0.15.1 (February 4, 2026) is the latest stable release PyPI GitHub and has resolved several historical issues: **SHA256 is now the default** prefix caching hash since v0.11.0, vLLM eliminating the CVE-2025-25183 hash collision vulnerability. GitHub PR #24954 (analysis channel content fix) was merged in v0.11.0. GitHub The tool-calling hang bug (#26480) was likely resolved through async scheduling improvements in v0.11.1+.

The fatal problem is that **GPT-OSS tool calling via `/v1/chat/completions` remains broken** with no fix planned. Issue #22578 is marked stale. GitHub The umbrella issue #23217 closed with 4 of 14 sub-tasks unfinished. No `tool-call-parser` value works for GPT-OSS — `hermes` fails to start, `mistral` and `llama3_json` produce mangled arguments. GitHub The `/v1/responses` endpoint handles Harmony format correctly, vLLM but LangChain's `ChatOpenAI` class exclusively uses Chat Completions. OpenAI GitHub Building a custom wrapper around the Responses API would mean abandoning LangChain's tool-calling abstractions, `bind_tools()`, and LangGraph's `ToolNode` — effectively rewriting the agent framework.

A secondary issue compounds the problem: **the hybrid attention prefix cache bug (#32802) is not fixed in v0.15.1**. When GPT-OSS runs with EAGLE3 speculative decoding, the `HybridKVCacheCoordinator` drops cache hits to 0%. The workaround (`--disable-hybrid-kv-cache-manager`) recovers ~80% hit rate but disables optimized hybrid cache allocation. Without speculative decoding, prefix caching works normally at ~88.7% hit rate. On Ampere hardware, vLLM uses the **TRITON_ATTN** attention backend and **Marlin MXFP4 MoE** kernels, vLLM both confirmed working on A100. vLLM

SGLang solves the three hardest problems simultaneously

SGLang v0.5.9 (February 23, 2026) PyPI addresses each pain point through architectural decisions rather than patches.

RadixAttention eliminates the entire class of hash-collision bugs. Instead of hashing token blocks into a lookup table, SGLang maintains a radix tree (compact prefix tree) where token sequences are stored as edge labels. LearnOpenCV Prefix matching is deterministic O(n) character comparison — no hashing, no collisions, no CVEs. Medium Cache-aware scheduling prioritizes requests by shared-prefix length, LearnOpenCV achieving **50-99% hit rates** across workloads. Sglang For GPT-OSS's hybrid attention pattern, RadixAttention caches the full-attention layers' KV normally while the sliding-window layers' minimal cache (128 tokens per layer) is handled through the attention backend configuration.

The `--tool-call-parser gpt-oss` flag produces standard OpenAI `tool_calls` objects. SGLang's `harmony_utils.py` translates raw Harmony tokens — `<|channel|>commentary to=functions.get_weather<|constrain|>json<|message|>{"location":"Tokyo"}<|call|>` — into proper Chat Completions format with `id`, `function.name`, and `function.arguments` fields. GitHub This is confirmed working in GitHub issue #10089, which shows the exact JSON output. github Combined with `--reasoning-parser gpt-oss` Sglang (which separates the analysis channel from user-facing content), Sglang the server launch command is:

bash

```
python3 -m sglang.launch_server \
  --model-path openai/gpt-oss-120b \
  --reasoning-parser gpt-oss \
  --tool-call-parser gpt-oss \
  --attention-backend triton \
  --tp 2 \
  --host 0.0.0.0 --port 30000
```

LangChain integration is then a drop-in:

```python
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(
    model="openai/gpt-oss-120b",
    base_url="http://localhost:30000/v1",
    api_key="dummy",
)
llm_with_tools = llm.bind_tools(tools)  # avoid tool_choice parameter
```

Langchain

Three known caveats require attention. First, passing tool_choice (e.g., tool_choice="required") crashes the HarmonyParser with NotImplementedError: structure_info not used with HarmonyParser GitHub — avoid this parameter in LangGraph tool nodes. Second, multiple system messages can cause issues (PR #13403 addresses this) GitHub — concatenate system prompts into one message. GitHub Third, if separate_reasoning is disabled, raw Harmony tokens like <|channel|> leak into responses GitHub — keep the default enabled via --reasoning-parser gpt-oss.

llama.cpp offers raw speed but can't solve the SWA-caching conflict

llama.cpp (rolling builds, ~b8099 as of February 2026, now under ggml-org) has **native GPT-OSS GGUF support with MXFP4** GitHub and delivers **10-20% higher single-user throughput** than vLLM on comparable hardware. The built-in gpt-oss chat template handles Harmony format, and llama-server exposes a /v1/chat/completions endpoint compatible with ChatOpenAI. Read the Docs

The fundamental problem is the **sliding-window attention metadata conflict with prefix caching**. When llama.cpp detects gpt-oss.attention.sliding_window=128 in GGUF metadata, it creates a small ring-buffer cache for SWA layers. This ring buffer is incompatible with prefix caching, which requires a linear buffer to store and reuse cached prefixes. The --swa-full flag forces the SWA cache to full size, GitHub enabling prefix caching but **dramatically increasing memory** — issue #14123 documents "very large RAM/VRAM usage." On a single A100-80GB already occupied by ~63GB of model weights, the additional memory for a full-size SWA cache alongside the ~4.5GB full-attention cache leaves almost no headroom. Issue #17118 (February 2026) shows GPT-OSS-20b still crashing with n_swa = 128, GitHub and issue #18497 confirms cache reuse remains ineffective for SWA models. GitHub

An additional limitation: **GBNF grammar constraints and tool calling cannot be used simultaneously** in llama.cpp because tool calling internally uses its own grammar. This means you cannot enforce valid Harmony JSON structure via GBNF while also using the built-in tool-calling path. You must choose one approach — the built-in template-based tool calling (which works but lacks grammar enforcement) or GBNF grammar (which requires manual Harmony grammar authoring and bypasses the tool-calling API).

128K context on single A100-80GB is the universal bottleneck

The memory arithmetic for GPT-OSS-120B on A100-80GB looks deceptively favorable: **~63GB model weights + ~4.5GB KV cache + ~3-5GB overhead = ~70-73GB**, leaving 7-10GB headroom. However, no framework vendor or community practitioner has demonstrated 128K context on a single A100-80GB in production. vLLM's official recipes show `--max-model-len 131072` only on 8×H100. (vLLM) LMCache confirms that single-GPU deployments have "less than 10GB KV cache buffer." (LMCache Blog) GPUStack benchmarks GPT-OSS at 32K context on 2×A100. (gpustack) The MXFP4 format was designed for (LM Studio) (vLLM) Hopper/Blackwell tensor cores; (vLLM) on Ampere, it runs through Marlin fallback kernels with additional overhead. (BytePlus)

The practical recommendation is **TP=2 across two A100-80GB GPUs** for reliable 128K context. On a single A100-80GB, cap context at **~32K-48K tokens** and use **LMCache CPU offloading** if longer contexts are occasionally needed (achieving ~67% TTFT reduction compared to recomputation). (lmcache) If FP8 KV cache support for GPT-OSS stabilizes in future vLLM/SGLang releases, that would halve the already-small 4.5GB cache — helpful but not transformative. (GitHub)

Performance expectations for single-user agent workloads

Precise benchmarks for GPT-OSS-120B on A100-80GB are sparse, but convergent data points allow reasonable estimates:

Metric	SGLang (projected)	vLLM (reported)	llama.cpp (projected)
Generation throughput (tok/s, single stream)	~40-55	~45-55	~50-65
TTFT at 32K context	~0.8-1.5s	~1.2-1.5s	~1.5-2.5s (no chunked prefill)
TTFT at 128K context (TP=2)	~3-6s	~4-8s	~8-15s
Prefix cache hit benefit	50-99% TTFT reduction	~80-89% reduction	Broken without `--swa-full`
Multi-turn agent latency (cached)	Best (RadixAttention + tree scheduling)	Good (with SHA256 cache)	Poor (SWA conflict)

SGLang's RadixAttention provides the largest advantage for the described LangGraph workload. (Sglang) In multi-turn agent conversations where each turn shares a long prefix with previous turns, RadixAttention's tree-

based scheduling achieves near-optimal cache reuse. Runpod Community benchmarks show SGLang delivers **3.7x faster TTFT than vLLM** at low concurrency GitHub (verified in SGLang's own benchmarks against vLLM v0.6.0, though the gap has narrowed in later versions). SGLang's per-token latency is also **more stable** at 4-21ms across loads, Clarifai compared to vLLM's higher variance. Clarifai

The recommended deployment configuration

Primary recommendation: SGLang v0.5.9, 2×A100-80GB, TP=2

```bash
python3 -m sglang.launch_server \
  --model-path openai/gpt-oss-120b \
  --reasoning-parser gpt-oss \
  --tool-call-parser gpt-oss \
  --attention-backend triton \
  --quantization mxfp4 \
  --tp 2 \
  --chunked-prefill-size 4096 \
  --mem-fraction-static 0.90 \
  --max-model-len 131072 \
  --host 0.0.0.0 --port 30000
```

Fallback for single A100-80GB (32K context limit):

```bash
python3 -m sglang.launch_server \
  --model-path openai/gpt-oss-120b \
  --reasoning-parser gpt-oss \
  --tool-call-parser gpt-oss \
  --attention-backend triton \
  --tp 1 \
  --chunked-prefill-size 2048 \
  --mem-fraction-static 0.92 \
  --max-model-len 32768 \
  --host 0.0.0.0 --port 30000
```

LangGraph integration (both configurations):

```python
```

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="openai/gpt-oss-120b",
    base_url="http://localhost:30000/v1",
    api_key="dummy",
    temperature=0.6,
    max_tokens=4096,
)
# bind_tools works; do NOT pass tool_choice parameter
llm_with_tools = llm.bind_tools(tools)
```

Upstream issues to watch for full resolution

- **SGLang `tool_choice` crash** (Roo-Code #10319): Blocks `tool_choice="required"` usage with HarmonyParser. Monitor sgl-project/sglang for a fix to `structure_info` handling. GitHub
- **SGLang Responses API custom functions** (#13967): Once resolved, enables both Chat Completions and Responses API paths for tool calling. GitHub github
- **vLLM Chat Completions GPT-OSS tool calling** (#22578, #23217): If vLLM adds a `gpt-oss` tool-call-parser, it would become viable again. GitHub GitHub Currently closed NOT_PLANNED.
- **vLLM hybrid attention cache** (#32802): Fix would restore full prefix cache performance with EAGLE speculative decoding.
- **llama.cpp SWA prefix caching** (#17118, #18497): Upstream resolution would eliminate the `--swa-full` memory penalty and make llama.cpp competitive for cached multi-turn workloads. GitHub GitHub
- **FP8 KV cache for GPT-OSS** (vllm #23832): Fix would provide additional memory headroom on single-GPU deployments. github

Conclusion

The inference framework landscape for GPT-OSS-120B in February 2026 has a clear hierarchy for LangGraph agent workloads. **SGLang v0.5.9 is the only framework where all four requirements are met without workarounds**: its `--tool-call-parser gpt-oss` flag produces standard Chat Completions `tool_calls`, Sglang LMSYS Org RadixAttention provides structurally sound prefix caching, LearnOpenCV Sglang and the A100 Triton backend handles MXFP4 MoE inference correctly. The 20% throughput gap versus llama.cpp is irrelevant when llama.cpp cannot deliver working prefix caching without crippling memory overhead, and the vLLM throughput comparison is moot when tool calling is broken at the API layer. GitHub The one compromise is hardware: full 128K context reliably requires two A100-80GB GPUs X with TP=2 across all frameworks, a constraint imposed by the ~63GB model weights vLLM rather than any framework limitation. BytePlus